BUSINESS SALES & PURCHASE FORECASTING SYSTEM

System Architecture and Technical Design

Purpose
A teaching prototype for exploring business time-series data, comparing forecasting methods, and presenting forecasts with limitations clearly.

Scope: CSV/XLSX ingestion • assisted column mapping • time-series preparation • ARIMA and Exponential Smoothing • chronological evaluation • Streamlit prototype

Architecture blueprint for the initial application phase

1. Executive Summary

The application turns uploaded business records into a reviewable time series and a forecast for one selected target: sales or purchases. Its central design principle is to keep forecasting logic independent from the Streamlit interface. The interface collects choices and presents results; a Python forecasting package validates the data, prepares the series, evaluates eligible methods, and returns forecasts together with diagnostics.

The prototype focuses on transparency rather than promising a forecast for every file. Column detection is assisted and confidence-aware. Ambiguous mappings require user confirmation. Models are compared using observations that occur after their training data. If the history is inadequate or candidate performance is weak, the application explains the limitation instead of presenting an unsupported prediction.

Intended audience: students learning end-to-end applied ML development, from requirements and architecture through implementation, evaluation, testing, and responsible use of coding agents.

2. Goals, Scope, and Boundaries

2.1 Goals

- Accept business history from CSV and Excel files without requiring one fixed spreadsheet schema.
- Identify likely date and monetary target fields, explain confidence, and allow user correction.
- Convert transaction records or summary rows into a validated time series at an inferred or user-confirmed frequency.
- Compare ARIMA and suitable Exponential Smoothing variants using chronological validation.
- Return forecast values, evaluation results, uncertainty information when supported, and data-quality warnings.
- Demonstrate the workflow through a small Streamlit interface while preserving a reusable engine boundary.

2.2 Initial scope

The first version is a local teaching prototype. It handles one uploaded dataset at a time, one forecast target per run, and a user-selected forecast horizon. Sales and purchases can be forecast in separate runs; if both targets are available, each receives its own preparation and model-selection path. The design supports later extension but does not assume production-scale hosting.

2.3 Out of scope for the initial phase

- Production multi-tenant SaaS, authentication, billing, and organization management.
- A permanent database, long-term file storage, background job queue, or distributed model-serving service.

- Deep learning, causal forecasting, external economic data, inventory optimization, or automatic business decisions.
- A guarantee that every upload can be forecast reliably.

3. Stakeholders and Primary Use Case

The primary user is a student or business analyst exploring historical records. The operator is responsible for confirming that the chosen date and target fields represent the intended business measure.

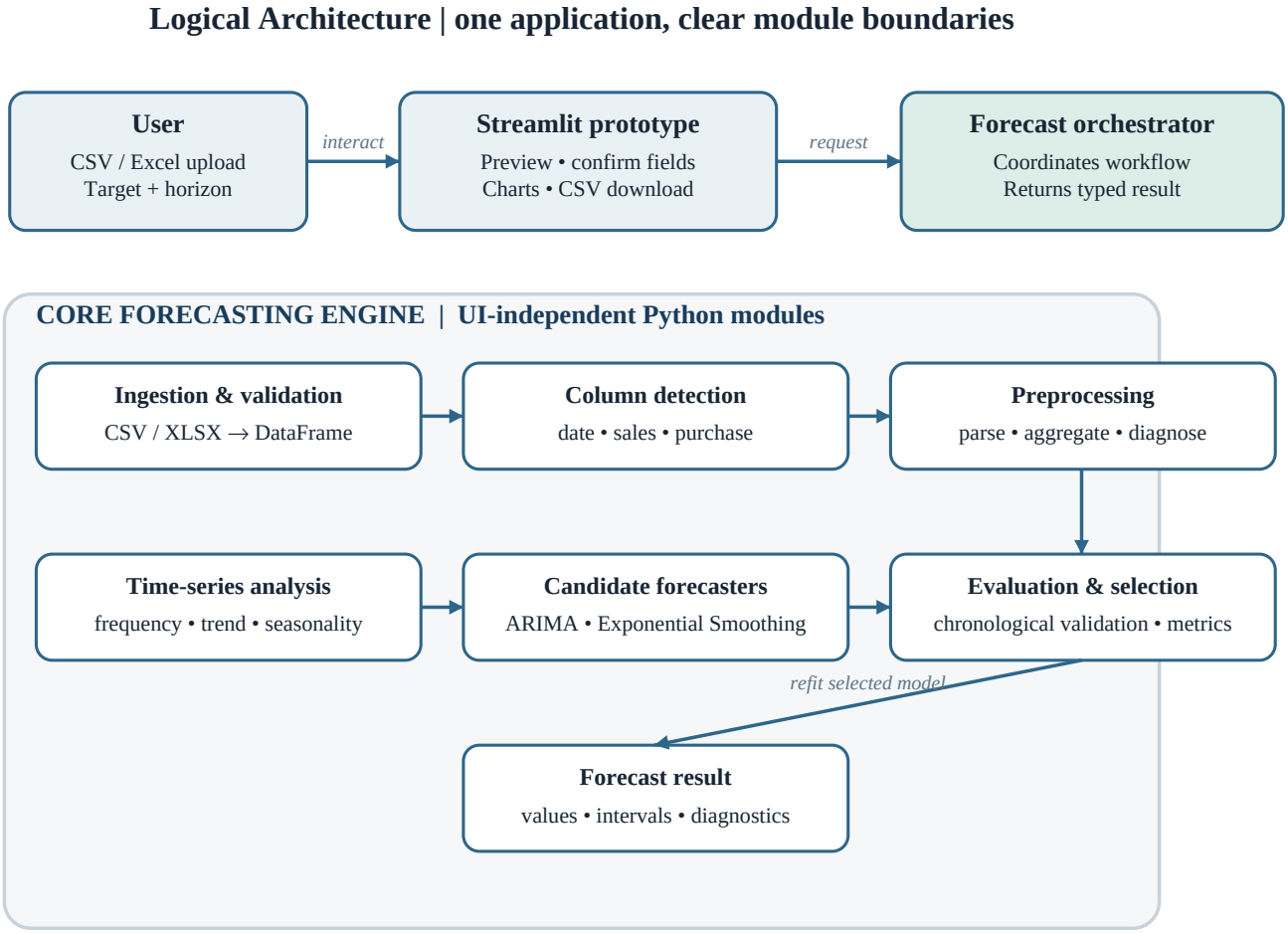
3.1 Main use case: forecast one business metric

- User uploads a CSV or Excel workbook and selects a sheet if needed.
- System previews the file, reports parsing issues, and proposes date and target columns.
- User confirms or corrects the mapping, aggregation meaning, and time frequency when needed.
- System prepares the series and displays diagnostics before fitting models.
- System evaluates eligible candidates on later historical periods and presents comparison results.
- User reviews the forecast, uncertainty bounds when available, warnings, and exports the result.

3.2 Key quality attributes

Quality	Design response
Correctness	Explicit mapping confirmation, validation rules, time-aware evaluation, and typed outputs.
Explainability	Show detected fields, preprocessing actions, candidate scores, metrics, and warnings.
Maintainability	Separate ingestion, detection, preprocessing, analysis, model fitting, evaluation, orchestration, and UI.
Reproducibility	Record configuration and library versions in run metadata; deterministic evaluation where possible.
Privacy	Process data locally and in memory by default; do not persist uploaded business data.

4. Logical System Architecture



No database or external service is required for the initial local prototype. Uploaded data is processed in-session.

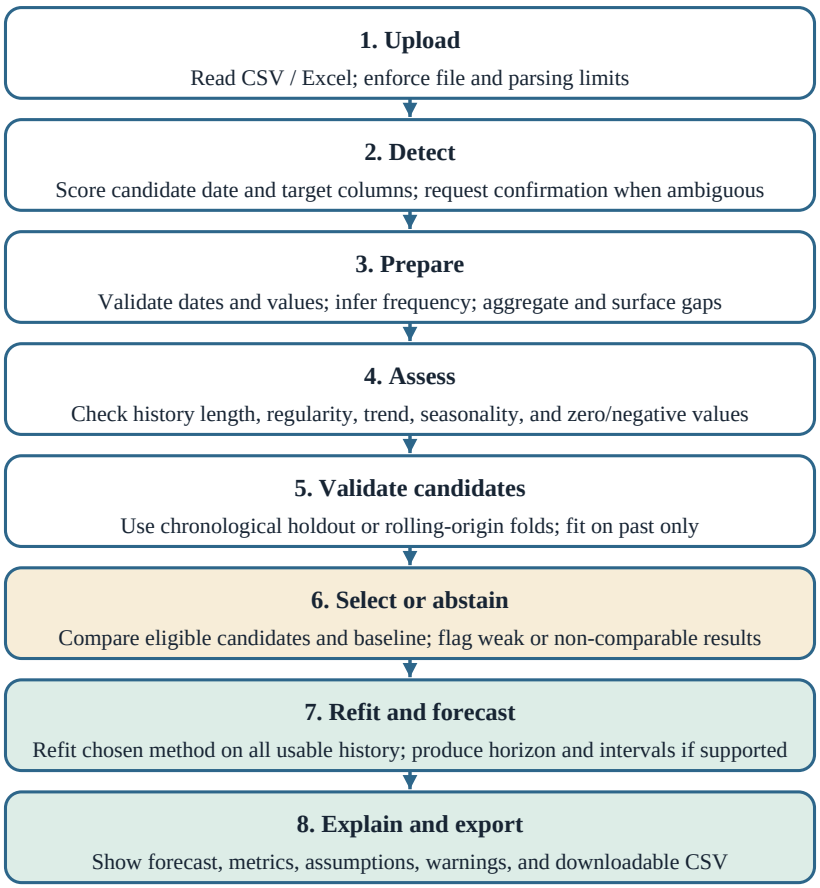
Figure 1. Logical architecture of the local prototype. The UI calls the orchestration boundary; forecasting modules do not import Streamlit. The forecast response contains both values and the evidence needed to interpret them.

4.1 Component responsibilities

Component	Responsibilities
Streamlit UI	File upload, data preview, field confirmation, target/horizon controls, result charts, warnings, and CSV export. No model logic.
Ingestion	Read CSV/XLS/XLSX, select a worksheet where relevant, validate file type/size/shape, and return a tabular object plus parsing diagnostics.
Column detection	Normalize column names, combine terminology with value/type evidence, score candidates, and expose ambiguous alternatives for confirmation.
Preprocessing	Parse dates and numbers, handle duplicate policy, aggregate transactions, infer/confirm frequency, represent missing periods, and emit a clean indexed series.
Time-series analysis	Describe history length, regularity, trend, seasonality evidence, variability, zero/negative values, and model eligibility.
Evaluation & selection	Create chronological folds, fit eligible candidates and baselines, calculate metrics, apply selection rule, and record failures.
Forecasting models	Fit ARIMA and supported exponential-smoothing variants behind a common interface. Return point forecasts and intervals when the implementation supports them.
Orchestrator	Validate request, sequence modules, enforce abstention/confirmation gates, assemble a result object, and translate expected failures into actionable messages.

5. End-to-End Data Flow

Forecast Request | end-to-end processing and decision points



User correction is required before forecasting whenever critical inputs remain ambiguous.

Figure 2. Request lifecycle. Data preparation and validation occur before model fitting. Confirmation gates prevent the application from treating ambiguous columns or missing intervals as settled facts.

5.1 Request and response boundary

The UI should call a single orchestration function with explicit inputs. A conceptual request contains the uploaded table, confirmed date column, selected target column, aggregation rule where relevant, frequency override if needed, forecast horizon, and evaluation configuration. The engine returns the resolved mapping, cleaned series metadata, diagnostics, candidate comparison, selected model (or no selection), forecast rows, and warnings.

Contract rule: the UI may display and collect values, but it must not perform hidden data cleaning or select models. The engine result should be serializable enough to support a future API without depending on Streamlit session objects.

5.2 User decision points

- Confirm date and target columns when detection is ambiguous.
- Confirm whether rows are transactions or already-periodic summaries.
- Review inferred frequency and aggregation method before forecasting.

- Accept that a forecast is unavailable when there is not enough usable history or no candidate passes the configured quality rule.

6. Data Contracts and Domain Objects

Use small, explicit objects at module boundaries. The following conceptual contracts clarify responsibility; implementation may use dataclasses, Pydantic models, or ordinary typed Python classes.

Object	Core fields	Purpose
UploadResult	table, source_name, columns, row_count, parse_warnings	Carries validated tabular input without embedding UI state.
ColumnCandidate	column_name, role, confidence, evidence, alternatives	Explains why a column is proposed and when confirmation is required.
SeriesData	target, frequency, values, index, aggregation, missing_periods, diagnostics	Represents one prepared univariate series and preparation provenance.
EvaluationResult	candidate_name, configuration, fold_metrics, aggregate_metrics, status, errors	Keeps successful and failed candidate attempts inspectable.
ForecastResponse	forecast_rows, selected_model, comparison, diagnostics, warnings, run_config	Complete, user-facing output of one forecast request.

6.1 Result status

Represent outcomes explicitly: success with a selected model; success with cautions; needs user confirmation; insufficient history; or no eligible model. Do not encode these distinct states as an empty forecast with no explanation.

6.2 Provenance

Record the resolved column mapping, frequency, aggregation rule, missing-period policy, evaluation windows, metric definitions, model configuration, and software version. This helps students reproduce and explain a forecast without retaining the original data file.

7. Column Detection and Preprocessing Design

7.1 Assisted column detection

Detection should combine normalized name matching (including configurable synonyms and abbreviations), inferred data type, parseable date proportion, numeric coverage, value ranges, and surrounding column context. Candidate ranking is evidence, not certainty. For example, a numeric field called Amount is not enough to distinguish sales from purchase cost.

The detector should return ranked candidates and the evidence behind each score. The interface requests confirmation if the best candidate is below a configurable threshold or close to another plausible candidate. Allow explicit user selection even when automatic confidence is high.

7.2 Transaction rows versus summarized observations

When each row is a transaction, aggregate the chosen monetary target by period using an explicitly selected rule, commonly a sum if that matches the business meaning. When each row already represents a period, avoid re-aggregation that would double-count. Ask the user or inspect date uniqueness and value patterns; expose the decision.

7.3 Missingness, duplicates, and negative values

- Do not replace absent periods with zero by default. Mark gaps and ask whether absence means no activity, unavailable reporting, or an unknown value.
- Define duplicate handling from available identifiers and business meaning; do not remove equal-valued rows blindly because two legitimate transactions may match.
- Keep valid zero observations. Retain negative adjustments, refunds, and returns unless the user selects a documented treatment.
- Report invalid dates, non-numeric target values, excluded rows, and every transformation that changes the series.

7.4 Frequency and sufficiency

Infer candidate frequency from date spacing and calendar alignment, then present it for confirmation when evidence is mixed. Forecast horizon is expressed in periods of that frequency. Model eligibility depends on the usable history length and seasonal period; the application must not fit seasonal models when the history cannot support them.

8. Forecasting and Model Selection

8.1 Candidate methods

Start with ARIMA and Exponential Smoothing variants supported by the series. A simple baseline, such as last value or seasonal-naive when enough seasonal history exists, is useful as a reference point. It helps answer whether a complex candidate adds value.

Use a shared forecaster interface for eligibility checks, fitting, prediction, and optional interval estimation. Keep candidate configuration bounded and documented; uncontrolled parameter search can be slow and can overfit small datasets.

8.2 Time-aware evaluation

Use chronological train/validation windows. For a sufficiently long series, rolling-origin evaluation can compare forecasts across multiple cut points. For short histories, use a clearly stated single holdout or declare that model comparison is unreliable. Every validation prediction must be made from data preceding its validation window; transformations and parameter choices must not inspect future validation values.

8.3 Metrics and selection rule

Metric	Interpretation	Use notes
MAE	Average absolute error in the target’s units.	Easy to interpret; use as a primary metric when units matter.
RMSE	Penalizes larger errors more strongly.	Sensitive to outliers; report alongside MAE.
MASE	Error scaled against a naive training-series forecast.	Useful for comparing series scales when denominator is valid.
sMAPE	Symmetric percentage-style error.	Can still behave awkwardly near zero; use carefully.

Choose and document a primary selection metric before evaluation. Compare candidates on matching folds and horizons. Do not use MAPE alone because zero or near-zero observations can make it undefined or misleading. Preserve fold-level metrics so unstable performance is visible.

8.4 Final forecast and uncertainty

After model selection, refit the chosen configuration on all usable history and generate the requested horizon. Return prediction intervals only where the method and implementation can support them; explain their meaning and assumptions. Intervals are not guarantees. If no candidate passes a configured adequacy criterion, return a clear no-reliable-forecast result rather than implying confidence.

9. Error Handling and Operational Behavior

Condition	System behavior
Unsupported file or corrupt workbook	Reject with supported formats and actionable parsing details; do not proceed with a partial silent read.
No usable date field	Ask user to select a valid date/time column; stop the forecast path.
Ambiguous target mapping	Show candidates and evidence; require user confirmation.
Too few observations	Explain which methods or comparisons cannot be supported; do not overstate forecast reliability.
Irregular or missing periods	Show detected gaps and ask/record a policy; do not silently convert gaps to zero.
Model convergence or fit failure	Record candidate failure, continue with other eligible candidates, and include the failure in diagnostics.
All candidates fail or perform poorly	Return an explicit abstention/weak-performance state with metrics and next steps.
Large or malicious upload	Enforce file size, row/column, and worksheet limits; handle parser exceptions safely.

9.1 Logging and observability

For the local prototype, use structured application logs for run identifiers, stage durations, selected configuration, and error categories. Do not log raw row contents or sensitive business values. In the UI, show progress by stage and concise warnings; reserve technical tracebacks for developer logs.

9.2 Privacy and temporary data

Default to in-memory processing in a local session. Avoid writing uploaded files or forecasts to disk unless explicitly required for export. Document that exports are user-initiated. If the application is later hosted, revisit access control, isolation between users, retention, encryption, file scanning, auditability, and deletion policies before deployment.

10. Deployment View

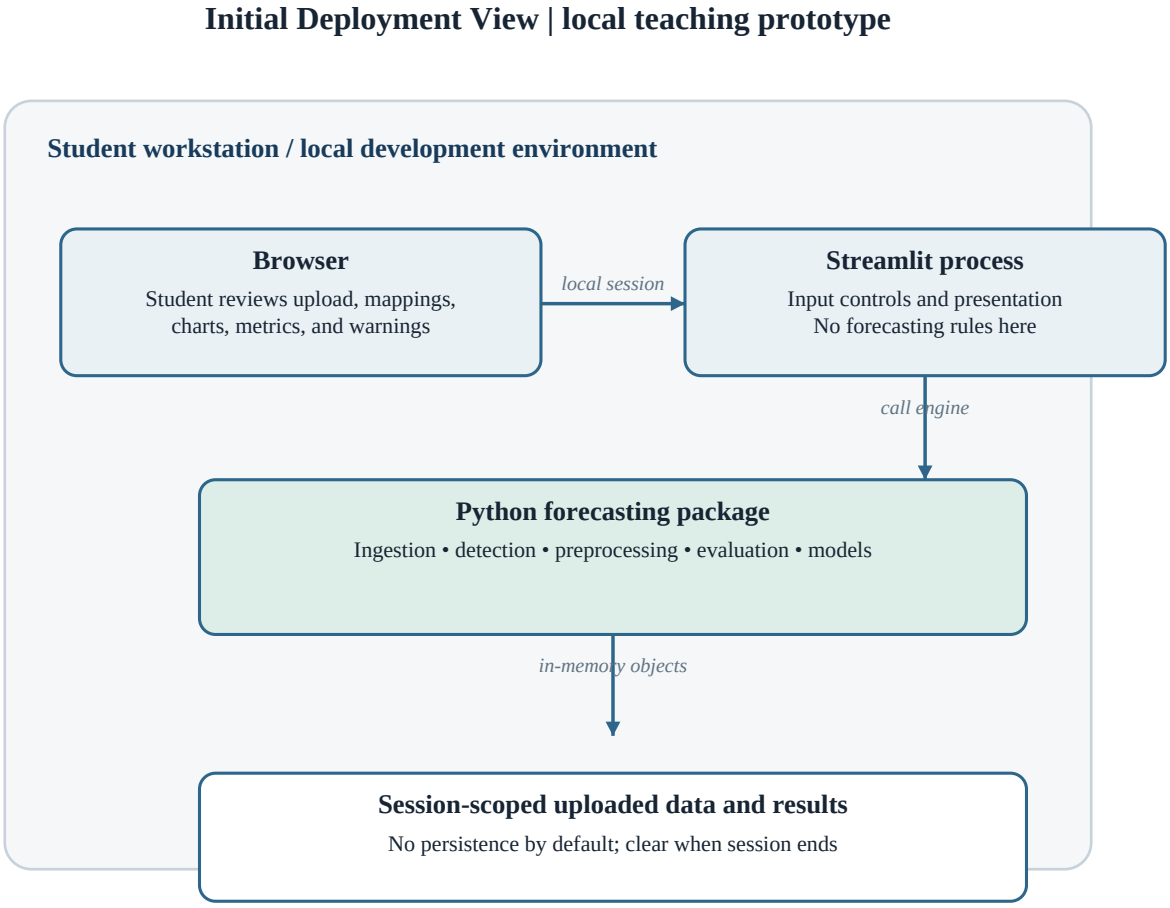


Figure 3. Initial deployment view. This is a single-user/local prototype with no durable storage in the default design. A hosted environment changes the trust boundary and requires additional decisions.

10.1 Local runtime

- Python environment with pinned dependencies and a reproducible setup file.
- Streamlit process runs locally; browser communicates with the local app session.
- Forecasting package operates on validated in-memory objects.
- Input datasets remain temporary by default; CSV forecast export is initiated by the user.

10.2 Later hosted deployment considerations

Before hosting for multiple organizations, define authentication and authorization, tenant isolation, upload limits and scanning, data retention and deletion, secure transport, secrets management, rate limits, resource limits, audit events, backup strategy, monitoring, and incident handling. These are future design requirements, not claims about the initial prototype.

11. Testing and Verification Strategy

11.1 Unit tests

- File readers: supported extensions, malformed files, worksheet selection, and size/shape limits.
- Column detection: synonym matching, numeric/date evidence, confidence ranking, ambiguous “amount” cases, and manual override.
- Preprocessing: parsing, aggregation, duplicate policy, missing periods, zero values, negative adjustments, and irregular frequency.
- Metrics: hand-checkable values including zero actuals and invalid scaled denominators.
- Forecasters: fit/predict contract, interval availability, and graceful candidate failure.

11.2 Integration and end-to-end tests

- A clean monthly transaction dataset with seasonal structure.
- A daily or weekly dataset with no seasonal evidence.
- Separate and combined sales/purchase target fields.
- Ambiguous columns that must stop for confirmation.
- Short, sparse, irregular, or low-quality history that should warn or abstain.
- At least one synthetic case where ARIMA wins and one where smoothing wins, without hardcoding which candidate must win for arbitrary real data.
- UI smoke test confirming the result metrics, forecast, warnings, and downloaded CSV match the engine response.

11.3 Acceptance criteria

A release candidate is acceptable when the user can upload a supported file, confirm the intended time and target fields, review preprocessing diagnostics, run chronological evaluation without data leakage, inspect model comparison and limitations, and export the forecast. Tests must demonstrate these behaviors and verify the no-forecast paths as carefully as the successful path.

12. Proposed Repository Structure

A small package layout keeps the engine independently importable while allowing the UI and tests to evolve separately.

Path	Responsibility
src/forecasting_app/ingestion.py	CSV/Excel reading and upload validation.
src/forecasting_app/detection.py	Column candidate scoring and confirmation metadata.
src/forecasting_app/preprocessing.py	Date/value cleanup, frequency handling, aggregation, and gap diagnostics.
src/forecasting_app/analysis.py	Series characteristics and eligibility checks.

src/forecasting_app/models/	Common forecaster contract plus ARIMA and smoothing adapters.
src/forecasting_app/evaluation.py	Chronological split/folds, metrics, and comparison records.
src/forecasting_app/service.py	Forecast orchestration and response assembly.
app.py	Streamlit-only user interaction and display.
tests/	Unit, integration, and app-flow tests.
sample_data/	Synthetic teaching datasets; never real client data.
pyproject.toml / requirements lock	Project metadata and reproducible dependency versions.

12.1 Dependency posture

Use a small set of established dependencies: pandas and an Excel reader for tabular input, statsmodels for initial statistical models, Streamlit for the prototype UI, and a test framework such as pytest. Pin and review versions. Add semantic/NLP packages only if a tested use case demonstrates a clear benefit over transparent heuristics and user confirmation.

13. Implementation Phases and Design Decisions

Phase	Deliverable	Exit check
1. Requirements & architecture	Use cases, boundaries, data contracts, diagrams, risk list.	Students can trace a request through the system and explain module ownership.
2. Ingestion & detection	File reader, preview data, candidate ranking, confirmation flow.	Ambiguous target never auto-selects silently.
3. Preparation & diagnostics	Time index, aggregation, gaps, data-quality report.	Synthetic edge cases behave according to documented policies.
4. Forecasting & evaluation	Naive baseline, ARIMA, smoothing, chronological metrics.	No random split; fold comparison is reproducible and traceable.
5. Orchestration & response	Stable request/response contracts and abstention states.	Engine callable without Streamlit.
6. Streamlit prototype	Upload, confirmation, results, charts, warnings, CSV export.	UI is a thin client of the engine.
7. Verification & handoff	Tests, setup instructions, architecture update, demo dataset.	End-to-end behavior and limitations are demonstrable.

13.1 Key decisions

- Use a modular monolith for the initial version: one deployable app with internal module boundaries.
- Use assisted detection with transparent confirmation rather than opaque auto-classification.

- Use a user-visible frequency and aggregation choice when inference is uncertain.
- Evaluate with chronological holdout or rolling-origin folds; never randomize time order.
- Include a baseline and allow an explicit no-reliable-forecast outcome.
- Avoid persistent storage until a real use case and privacy policy justify it.

13.2 Important trade-offs

Automatic inference improves convenience but can be wrong when spreadsheet labels or business semantics are unclear. More candidate models can improve coverage but increase runtime and overfitting risk. Prediction intervals communicate uncertainty but depend on model assumptions and data quality. The prototype should expose these trade-offs rather than hide them.

14. Risks, Assumptions, and Open Questions

Risk / assumption	Mitigation or decision needed
Column name does not reveal business meaning.	Show ranked candidates, evidence, and require confirmation when uncertain.
Missing period may mean zero activity or missing reporting.	Ask or preserve as missing; record chosen policy.
Different business data has different aggregation semantics.	Confirm transaction vs summary data and aggregation rule.
Short history cannot support seasonal estimation or fair model ranking.	Check eligibility and communicate weak evidence.
Model can win validation but still be unreliable in future.	Show fold variability, baseline comparison, intervals when supported, and caveats.
Excel files may be large or malformed.	Set file and worksheet limits; catch errors and provide safe feedback.
Local prototype may be mistaken for production-ready software.	Document non-goals and list hosted deployment controls as future work.

14.1 Assumptions to confirm during implementation

- Whether uploads may contain multiple worksheets and how users select one.
- Which aggregation choices are required for transaction-level amounts.
- Whether users should forecast sales and purchases in one run or one target at a time.
- What adequacy threshold should trigger a warning or abstention.
- Whether prediction intervals are required for both initial model families.

15. Final Architecture Summary

The recommended first release is a local, single-user modular application. Streamlit handles interaction; a UI-independent Python engine owns file validation, mapping, time-series preparation, candidate evaluation, model selection, and forecasting. The engine treats column detection as evidence-based assistance, uses chronological out-of-sample evaluation, and returns diagnostics alongside forecasts. It can explicitly request user input or decline to make a forecast when data quality or history is insufficient.

This architecture gives students a practical path through system design, implementation, testing, and AI-assisted development while keeping the project small enough to understand. Future APIs, hosted deployment, persistent storage, or additional models can be added after the core contracts and forecasting behavior are demonstrated.